

# AICE2008 RISC-V CPU Coursework

September 30, 2026

## Abstract

This report documents the design and simulation of a single cycle RISC-V (RV32I) CPU, implemented in SystemVerilog and verified using testbenches simulated in Vivado. The core datapath includes the program counter, instruction memory, register file, ALU, immediate generator, control unit, multiplexers and data memory. Both module-level and top-level testbenches were used to validate functionality. Most base integer instructions were implemented, excluded instructions include ecall, ebreak, fence, fence.i, and byte/halfword load-store variants (while lw and sw were implemented). Simulation results for the implemented instruction set matched expected behavior, demonstrating correct datapath operation for the tested cases

## 1 Introduction

This project is about designing and testing a single-cycle RISC-V CPU using SystemVerilog, putting into action the fetch-decode-execute behaviour. The work is motivated by the need to connect instructions with datapath and control design at an architectural level in hardware design.

The aim is to design a CPU capable of executing a subset of RV32I instructions (the list of which can be seen in 1), and to be able to run simple programs loaded into the instruction memory.

In this report, the background section covers relevant knowledge relating to RV32I and single-cycle concepts, the method section explains my module design and integration decisions, the results section provides simulation evidence from testbenches, and the discussion/conclusion evaluate correctness of the designed system, any limitations, and further improvements that could be made.

| Category | Instructions                                         |
|----------|------------------------------------------------------|
| R-type   | add, sub, and, or, xor, slt, sltu, sll, srl, sra     |
| I-type   | addi, andi, ori, xori, slti, sltiu, slli, srli, srai |
| Branch   | beq, bne, blt, bge, bltu, bgeu                       |
| Jump     | jal, jalr                                            |
| Upper    | lui, auipc                                           |
| Memory   | lw, sw                                               |

Table 1: Instruction Categories

## 2 Background

The RV32I instruction set is a great foundation for developing a single-cycle CPU because it includes the essential instruction classes for arithmetic, control flow, and memory access. In a single-cycle implementation, all stages of instruction execution (fetch, decode, execute, memory access, and write-back) are completed within one clock period. This simplifies the design, though greatly limits the efficiency of the design due to the cost of a longer critical path. Understanding how instruction formats work is essential to this project, as the different instruction types require different immediate generation, ALU behaviour, branch/jump target calculation, and memory/register control signals. Though being a widely available, open-source architecture, there are many useful resources on understanding instructions and datapaths, I found the official docs (at <https://docs.riscv.org/>) in particular to be very helpful.

The datapath for RV32I CPU usually includes the program counter, instruction memory, register file, immediate generator, arithmetic logic unit (ALU), data memory and multiplexers (for selecting next pc and write back sources).

The datapath is as follows, the program counter provides the instruction address, instruction memory returns the encoded instruction, and the register file supplies source operands while capturing destination results. Then the immediate generator extracts the instruction constants, and the ALU performs arithmetic, logical, comparison, and address calculations. For load and store instructions, data memory is accessed using the ALU addresses, and the write back path selects either ALU results, loaded data, or specific instruction values. This datapath forms the structure to be able to execute simple programs stored in instruction memory and data operations through data memory.

To be able to implement this, instructions need to be correctly decoded into opcodes and function fields, then coordinated into control signals each cycle. This is the job of the control unit, it determines register write enable, memory read/write behaviour, ALU operation, branch/jump, and write-back source selection. Instruction coverage however, tends to be less fixed in comparison, and is a design choice, and while some are mandatory for meaningful functionality (notably memory operation), many may be omitted depending on scope of a given project. The specification also explicitly excludes fence, fence.i, ecall, ebreak, and CSR instructions, which defines a clear boundary for implementation and evaluation in this coursework.

### 3 Method

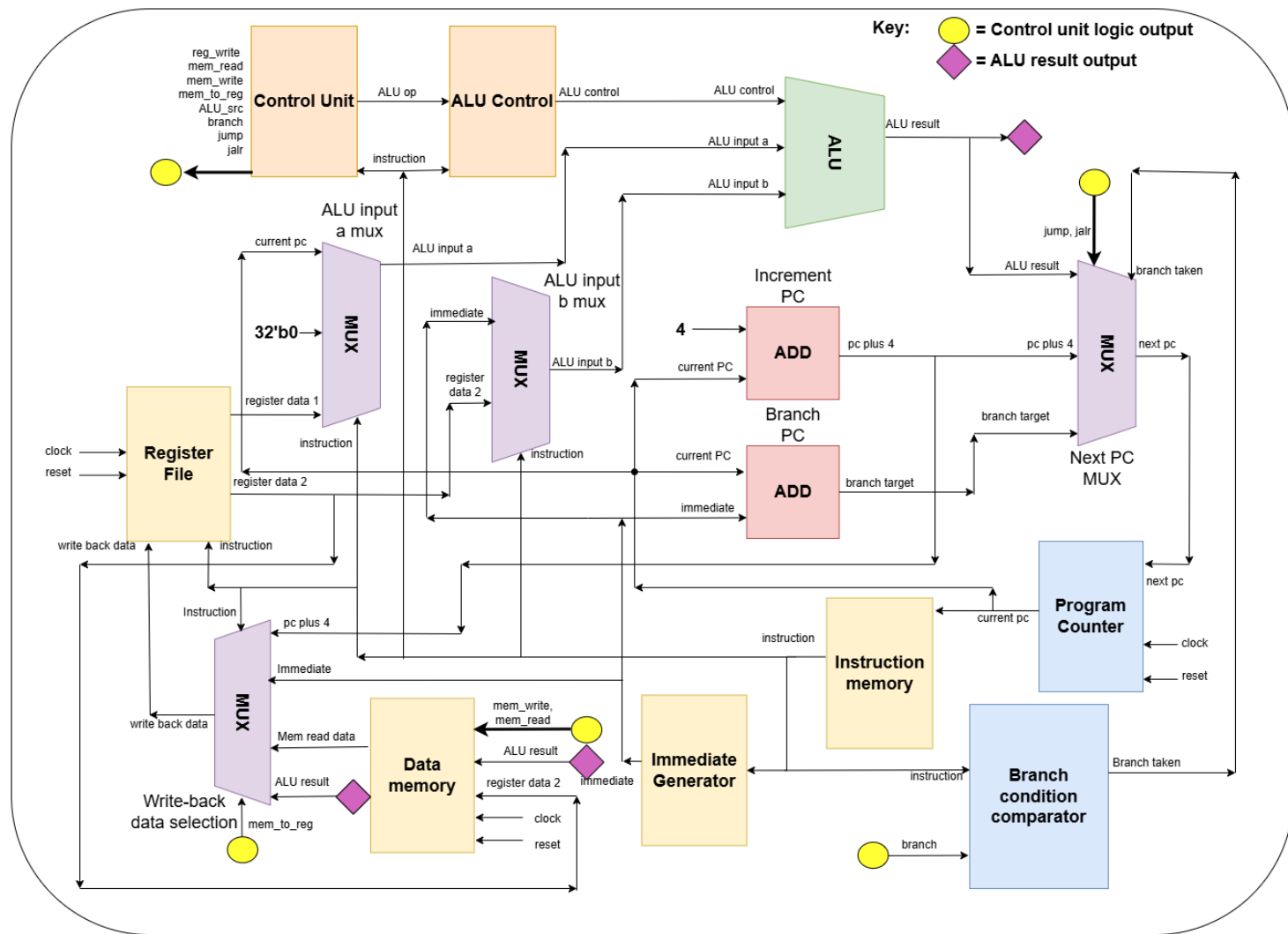


Figure 1: Diagram of RISC-V 32I CPU

I developed the RV32I processor in an iterative process, I began by planning out the different modules, and what inputs and outputs they need to complete the core datapath, 1 shows the flow of data through the different modules that I decided on. The main difference between the one datapath I described above and the one I implemented is the distribution of control logic across multiple modules, I broke control unit logic down into a Control unit and

ALU control module. I did this to reduce complexity, and allow for more focused unit tests of each module, instead of one testbench for all the different facets of the control unit.

After planning out the core datapath, I started with getting a bare minimum ADDI program to run through the whole datapath, then incrementally adding new groups of instruction and their associated glue logic, updating the testbenches throughout development. I also maintained a definitions file which I used to store values for opcodes, funct3 codes, funct7 codes, ALUOp codes (Control unit to ALU control) and ALU internal op codes (ALU control to ALU), this helped with clarity of code and debugging, as I didn't have to rely on memorising binary codes.

### 3.1 Program Counter

The role of the program counter is to return the address of the next instruction to be executed. It gets this address from the Next PC MUX, and so, does not have to carry out any computations for finding the next address. I have additionally added reset functionality, so on the posedge of reset, the program counter will be set to 32'b0.

### 3.2 Instruction memory

The purpose of the instruction memory module is to maintain an internal block of memory, and when given an address, it should return the value stored in that address space. As the address uses word indexing, I had to use `address[31:2]` to refer to it. I used a memory with 255 instruction slots as this would be larger than any program I'd have to run for testing purposes.

### 3.3 Immediate generator

The immediate generator produces the immediate value when given an instruction. It's important because depending on the instruction type, there will be a different order in which to extract immediate values out of the instruction. The different instruction types accounted for are R-type, I-type, S-Type, B-type, J-type and U-type.

### 3.4 Register file

The register file handles reading and writing data to the 32, internally managed registers. It takes in the clock, reset, reg write, rs1, rs2, rd, and write data, this sends out read data1 and read data2. In combinatorial logic, read data 1 and 2 will be set to the value of registers at rs1 and rs2 respectively (the register values of the instruction), unless they are referring to x0, in which case they are hard coded to return 0. Then on the positive edge of the clock, if reset is high, the values of each register will be set to 0, then when you are writing if the register isn't x0, then the new write data is written to the specified position at rd.

### 3.5 Data memory

Data memory is quite similar in function to the register file, the main differences being the size of the internal data, the input and output variables, and that you can write to the first position of the memory array. It takes in clock, reset, mem read, mem write, ALU result, reg data 2 and read data mem. In the combinatorial logic, if mem read is logic high, then read data mem will be set to the value at the specified address in the internal memory array (256 addresses with 32 bits of data each), otherwise, read data mem will be set to 0. Then in the always ff block, on the posedge of the clock, if mem write is high, then write data will be pushed to the currently addressed memory location. If reset is high, all memory locations will instead be set to 0.

### 3.6 Control Unit

The control unit, in essence, determines the properties of the supplied opcode, these properties include reg write, mem read, mem write, mem to reg, ALU src, branch, jump, jalr, and ALU op. It does this solely through combinatorial logic by checking what group of instructions (that share properties), the given opcode is a part of, then output the properties of that group. These groupings are R-type, immediate, load, store, branch, jal, jalr, lui and auipc. ALU op gives the rough category of the operation so that the ALU control narrows it down further these rough categories are R-type, I-type, branch and add, with add being the default for most operations.

### 3.7 ALU control

ALU control works out the specific operation the ALU needs to carry out for the current instruction by comparing the values for ALU op, funct3, and funct7 against the values in Definitions for the instructions. This essentially works as a series of case statements to determine the value for ALU control.

### 3.8 ALU

The ALU is the module that executes the given instruction to find the answer. It takes in the values in two registers, then depending on the instruction given by the ALU control, will carry out a mathematical operation and return the result of the two values.

### 3.9 Top

The top module contains the entire RISC-V CPU. It handles the internal logic that is passed between the various modules it instantiates, as well as defining the branch condition comparator, 4 muxes, and 2 adders.

The branch condition comparator takes in branch and funct3, if branch is true, then it will determine what type of branch instruction is being called, and will do the according comparison with reg data 1 and 2 to see if a branch needs to be taken.

There are two adders in the top, the first one simply increments the current PC by 4 (equivalent to 4 bytes or one word), while the other adds the immediate value onto the PC instead (branch target). Both of these send their outputs to the next PC MUX, in addition to ALU result, then if the instruction was a jump and jalr, then next pc becomes the ALU result rounded down to the next even number, to ensure the minimum byte alignment for jalr. Then if jump or branch taken are true, then the branch target is taken instead, otherwise the next PC will be the previous PC plus 4.

Finally there are 3 other MUXes in the circuit, the first 2 are for selecting the A and B values for the ALU. For input A, if the opcode is for AUIPC, then A becomes the current pc, if the opcode is LUI, A is set to 32'b0, then for all other operation, A is reg data 1. Then for input B, if ALU src is true, B will be the immediate, otherwise it is reg data 2. The last MUX determines what line to set write back data to for the register file. If the opcode is JAL or JALR, then write back data is the current PC plus 4, if the opcode is LUI, then it will be immediate, if mem to reg is high, write back data is mem read data, otherwise, write back data will be set to ALU result.

### 3.10 Testbenches

For the testbenches on the individual modules, I followed a similar test pattern, where I would create a check\_signals task with a test name and the module outputs I expect to see at a given point. The task then checks the input parameters with the real current parameters, and if any of them do not match the real values, then the test is output to have failed to the console, otherwise it says it passed; this allows robustness in testing and much less redundant code. Every source file has an associated testbench.

The top testbench differs quite a bit from the other testbenches, for it, I have designed 7 .mem files with hex programs, the assembly and expected results can be found in Test Files explained.txt; note that I converted the SIMPLE\_PROGRAM.mem to RISC-V hex instructions but for the rest of the .mems I used [this website](#) to automatically convert the assembly to hex due to how error prone and time consuming converting instructions by hand can be. These programs are run sequentially, with the values in each register displayed after each program, as well as the state of the program after each instruction.

SIMPLE\_PROGRAM.mem was the first program I designed to ensure the core functionality was correct, then I made separate test files for R-type, upper-imm, memory, jump, branch and I-type instructions, these test multiple cases for each instruction to ensure strong validation. Though I did not create automated tests to display passes and fails, I did manually check over the final register values after running it and found they all matched the expected values defined in Test Files explained.txt.

## 4 Results

All of the module-level automated testbenches, after cleaning up any errors that arose, return a 100% pass rate. This includes those for ALU\_Control\_TB, ALU\_TB, Control\_Unit\_TB, Data\_Memory\_TB, Immediate\_Generator\_TB,

Instruction\_Memory\_TB, Program\_Counter\_TB, Register\_File\_TB. Furthermore, you can see in ?? a selection of the more complex instructions across the .mem files, where all of them resulted in the expected result.

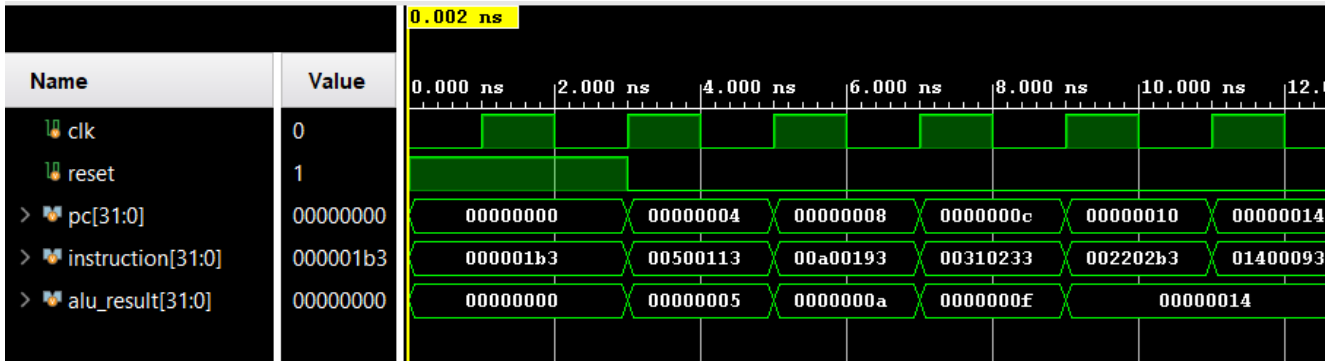


Figure 2: Start of the Top testbench, you can see the standard operation of PC incrementing correctly, as well as the instructions being read from the memory file

Listing 1: ALU testbench

```
Starting ALU testbench
PASS: ADD
PASS: SUB to zero
PASS: AND
PASS: OR
PASS: XOR
PASS: SLL
PASS: SRL
PASS: SRA
PASS: SLT signed
PASS: SLTU unsigned
ALU testbench finished
```

Listing 2: Control Unit

```
Starting Control_Unit testbench
PASS: R-type
PASS: I-type
PASS: LOAD
PASS: STORE
PASS: BRANCH
PASS: JAL
PASS: JALR
PASS: LUI
PASS: AUIPC
PASS: DEFAULT
Control_Unit testbench finished
```

Listing 3: ALU Control

```
Starting ALU_Control testbench
PASS: ALUOP_ADD -> ALU_ADD
PASS: BRANCH BEQ -> SUB
PASS: BRANCH BNE -> SUB
PASS: BRANCH BLT -> SLT
PASS: BRANCH BGE -> SLT
PASS: BRANCH BLTU -> SLTU
PASS: BRANCH BGEU -> SLTU
PASS: RTYPE ADD
PASS: RTYPE SUB
PASS: RTYPE AND
PASS: RTYPE OR
PASS: RTYPE XOR
PASS: RTYPE SLT
PASS: RTYPE SLTU
PASS: RTYPE SLL
PASS: RTYPE SRL
PASS: RTYPE SRA
PASS: ITYPE ADDI
PASS: ITYPE ANDI
PASS: ITYPE ORI
PASS: ITYPE XORI
PASS: ITYPE SLTI
PASS: ITYPE SLTIU
PASS: ITYPE SLLI
PASS: ITYPE SRLI
PASS: ITYPE SRAI
ALU_Control testbench finished
```

Figure 3: ALU, Control Unit, and ALU Control testbench outputs all passing

## 5 Discussion

### 5.1 What went well

The iterative development helped greatly with programming the system. After the minimum data path was created, it was easy to implement new instruction groups then test and validate them before moving onto the next feature. Separating the decoding functionality into a control unit and ALU control module reduced design complexity and made debugging more manageable through focused unit testing.

### 5.2 Key challenges and solutions

An issue I did run into while coding the immediate generator was with the padding on immediate values, I initially programmed the immediate to just be the extracted immediate value and rest 0s, but this led to issues with negative

Table 2: Representative instruction tests.

| Entry A |                                                                                                                                           |                                                                    |                                                                    |   | Entry B      |                                                                                                                                          |                                                                    |                                                                    |   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|---|--------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|---|
| Type    | Asm                                                                                                                                       | Exp.                                                               | Act.                                                               | P | Type         | Asm                                                                                                                                      | Exp.                                                               | Act.                                                               | P |
| ADDI    | <code>addi x3,x0,10</code>                                                                                                                | <code>x3=10</code>                                                 | <code>x3=10</code>                                                 | Y | JAL          | <code>jal x1,t;</code><br><code>addi x2,99;</code><br><code>t:addi x3,2</code>                                                           | <code>x1=ret;</code><br><code>x2 skip;</code><br><code>x3=2</code> | <code>x1=ret;</code><br><code>x2 skip;</code><br><code>x3=2</code> | Y |
| JALR    | <code>auipc x4,0;</code><br><code>addi x4,16;</code><br><code>jalr x5,0(x4);</code><br><code>addi x6,99;</code><br><code>addi x7,7</code> | <code>x5=ret;</code><br><code>x6 skip;</code><br><code>x7=7</code> | <code>x5=ret;</code><br><code>x6 skip;</code><br><code>x7=7</code> | Y | BEQ          | <code>beq x1,x2,L;</code><br><code>addi x3,-1;</code><br><code>L:addi x3,1</code>                                                        | <code>x3=1</code><br><br><code>x3=1</code>                         | <code>x3=1</code><br><br><code>x3=1</code>                         | Y |
| BEQ     | <code>beq x1,x0,fail;</code><br><code>addi x4,1</code>                                                                                    | <code>x4=1</code>                                                  | <code>x4=1</code>                                                  | Y | BLT          | <code>blt x10,x11,L;</code><br><code>addi x7,-1;</code><br><code>L:addi x7,1</code>                                                      | <code>x7=1</code><br><br><code>x7=1</code>                         | <code>x7=1</code><br><br><code>x7=1</code>                         | Y |
| BLT     | <code>blt x11,x10,fail;</code><br><code>addi x8,1</code>                                                                                  | <code>x8=1</code>                                                  | <code>x8=1</code>                                                  | Y | LUI          | <code>lui x1,0x12345;</code><br><code>lui x2,0</code>                                                                                    | <code>x1=12345000;</code><br><code>x2=0</code>                     | <code>x1=12345000;</code><br><code>x2=0</code>                     | Y |
| AUIPC   | <code>auipc x3,0;</code><br><code>auipc x4,4</code>                                                                                       | <code>x3=PC;</code><br><code>x4=PC+4000</code>                     | <code>x3=PC;</code><br><code>x4=PC+4000</code>                     | Y | LW/SW        | <code>x1=0;</code><br><code>x2=123;</code><br><code>sw x2,0(x1);</code><br><code>lw x3,0(x1)</code>                                      | <code>x3=123</code>                                                | <code>x3=123</code>                                                | Y |
| LW/SW   | <code>x4=-1;</code><br><code>sw x4,4(x1);</code><br><code>lw x5,4(x1)</code>                                                              | <code>x5=ffffff</code>                                             | <code>x5=ffffff</code>                                             | Y | LW/SW        | <code>sw x0,8(x1);</code><br><code>lw x6,8(x1);</code><br><code>addi x31,1</code>                                                        | <code>x6=0;</code><br><code>x31=1</code>                           | <code>x6=0;</code><br><code>x31=1</code>                           | Y |
| SLT     | <code>x1=-1;</code><br><code>x2=1;</code><br><code>slt x18,x1,x2;</code><br><code>slt x19,x2,x1;</code><br><code>slt x20,x1,x1</code>     | <code>x18=1;</code><br><code>x19=0;</code><br><code>x20=0</code>   | <code>x18=1;</code><br><code>x19=0;</code><br><code>x20=0</code>   | Y | SLTU         | <code>x1=-1;</code><br><code>x2=1;</code><br><code>sltu x21,x1,x2;</code><br><code>sltu x22,x2,x1;</code><br><code>sltu x23,x1,x1</code> | <code>x21=0;</code><br><code>x22=1;</code><br><code>x23=0</code>   | <code>x21=0;</code><br><code>x22=1;</code><br><code>x23=0</code>   | Y |
| SRA     | <code>lui x1,0x800000;</code><br><code>addi x2,31;</code><br><code>sra x29,x1,x2;</code><br><code>sra x30,x1,x0</code>                    | <code>x29=-1;</code><br><code>x30=80000000</code>                  | <code>x29=-1;</code><br><code>x30=80000000</code>                  | Y | <i>spare</i> |                                                                                                                                          |                                                                    |                                                                    |   |

numbers, as they would become large positive numbers, for example, -1 would become 2,147,483,646. Fixing this required filling the padding with the correct sign extension instead of just 0s. Additional complexity came about in control flow behaviour, particularly branch decision handling and JALR target formation, where careful validation of PC updates was required.

### 5.3 Current limitations

Although simulation results were strong for the implemented instruction groups, there were clear limitations with the design. The main one being the single-cycle processing restricting performance with long combinatorial critical paths. Furthermore, while the module testing has automatic validation, the top file testbench did not, and so it is less scalable if I were to add more mem files.

If I were to continue developing this, I would complete the RV32I instruction set, then move on to the standard extensions, such as RV32M, RV32A, or RV32F. I would also like to take steps towards a pipelined design to optimise the CPU.

## 6 Conclusion

Overall, I achieved the main goal of the coursework by designing and simulating a working single-cycle RV32I CPU in SystemVerilog. The main datapath and control modules were integrated successfully, and simulation results matched expected behavior for the instruction groups I implemented (arithmetic, immediate, memory, branch, and jump).

Even though the design does not cover every possible instruction and still has single-cycle timing limitations, it provides a solid functional base. The project gave me a much better understanding of processor implementation and a clear path for future improvements in verification, instruction coverage, and architecture.

## 7 References

Assembly to RISC-V instructions website: <https://riscvasm.lucasteske.dev/>  
RISC-V docs: <https://docs.riscv.org/reference/isa/unpriv/rv32.html>